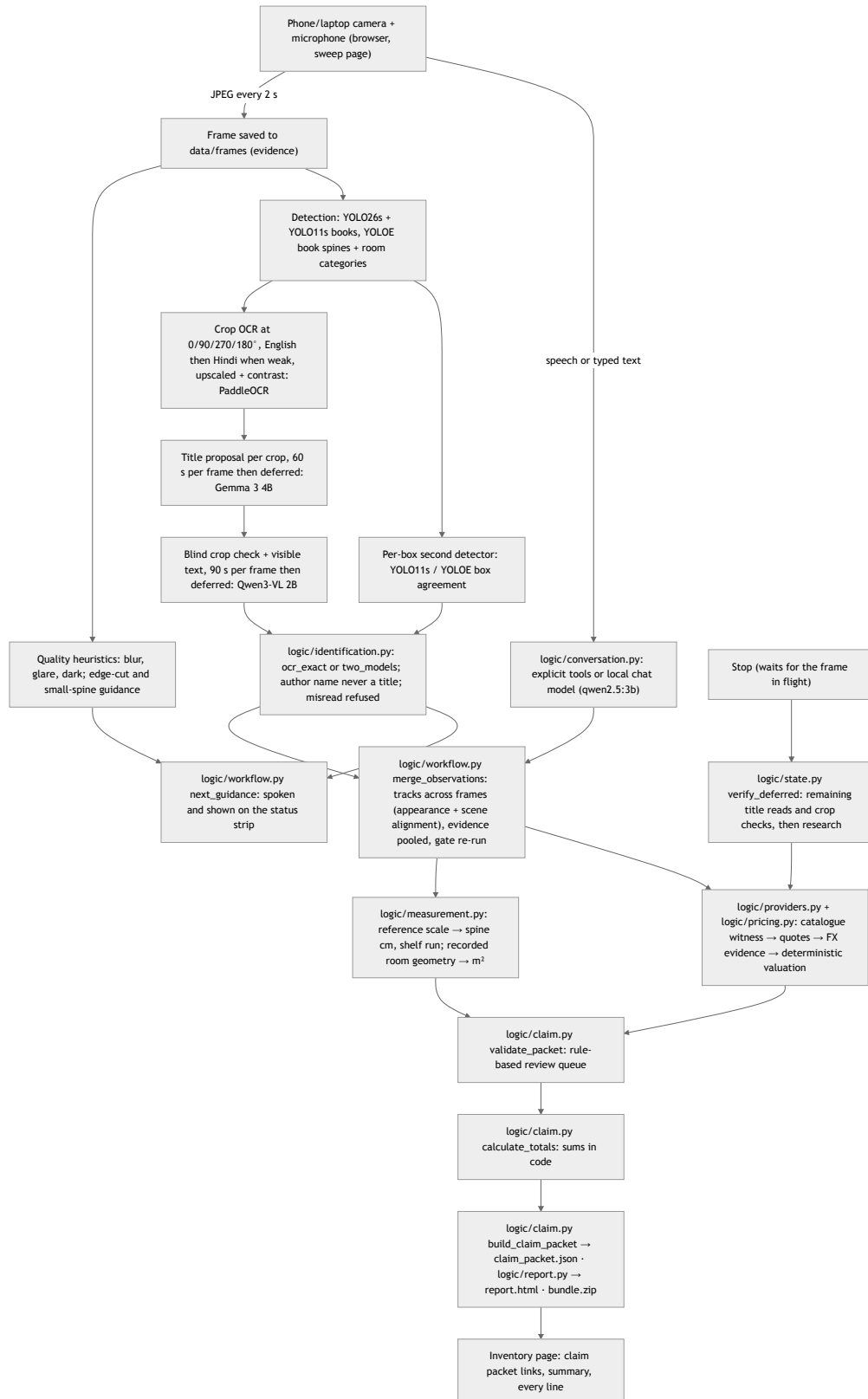


Architecture note

Author: Pankaj Yadav, Forward Deployed Software Engineer · github.com/Pankkaj64 · linkedin.com/in/pankkaj64 · resume: docs/pdf/Pankaj-Yadav-Resume.pdf

One continuous sweep, separate testable stages, no invented numbers.



Which model does what

Stage	Component	Role	Can it write a fact?
Capture quality	<code>logic/perception.py</code>	Pillow heuristics for blur, glare, darkness, resolution → spoken guidance	no (review notes)
Book / object boxes	YOLO26s + YOLO11s (<code>perception.detect_objects</code>) and YOLOE "book" boxes	localizes books and COCO objects; every detector that drew a box is recorded in <code>seen_by</code>	boxes only
Room categories	YOLOE26s with fused prompts (classes in <code>config.py</code>)	shelving, lamp, framed painting, rug, coffee machine, ...	category <i>proposal</i>
Second count	YOLO11s (<code>validate_boxes</code>)	per book: another detector drew the same box ($\text{IoU} \geq 0.5$) → <code>count_verified</code> ; frame-wide count equality is only a review note	agreement flag
Spine text	PaddleOCR / EasyOCR (<code>logic/ocr.py</code>)	text lines + confidence per crop, best of four rotations; English, then Hindi (Devanagari) when the English read is weak	OCR evidence
Title proposal	Gemma 3 4B via Ollama (<code>read_visual_title</code>)	reads the visible title; author/publisher constrained to OCR lines	proposal only
Crop check	Qwen3-VL 2B via Ollama (<code>logic/crop_verifier.py</code>)	blind category check of the crop without the detector label	agree / conflict
Identity	<code>logic/identification.py</code> (pure rules)	accepts a title only when OCR text (≥ 0.9), reader title, blind crop check and the per-book second-detector witness agree	yes (status identified)
Catalogue	Open Library (<code>logic/identification.py</code>)	resolves a work; edition only from a checksum-valid visible ISBN	work / edition metadata
Conversation	qwen2.5:3b via Ollama (<code>logic/conversation.py</code>)	answers questions with the live state; explicit commands bypass it	no

Why small local models, not bigger ones

The system is built and measured on an **Apple M3 laptop with 8 GB of memory**, and the brief expects a reviewer to run it from the README on their own laptop. On Apple silicon the CPU and GPU share that 8 GB, so every model competes with macOS, the browser and the backend.

What is loaded	Memory
Gemma 3 4B (title reader)	3.3 GB
Qwen3-VL 2B (blind check)	1.9 GB on disk, 2.7 GB once loaded
Qwen2.5 3B (conversation)	1.9 GB
Three YOLO detectors	about 70 MB
PaddleOCR, English and Hindi	about 220 MB
macOS, the browser and the backend	the rest of the 8 GB

- **A bigger vision model does not fit.** Qwen2.5-VL 3B, the first choice, cannot load: Ollama's Metal backend asks for a 6.6 GB buffer. With a 1k context it loaded but took 57–160 s per crop. 7B and larger vision models need more memory than the machine has.
- **The small models already fill the memory.** Together they need about 7 GB, so macOS swaps when all three are active. That is why OCR took 107 s for one frame inside the app but 2–8 s per crop on its own, and why each model gets a per-frame time budget.
- **Hosted large models were rejected on purpose.** A cloud model (GPT-4o, Gemini, Claude) would read worn spines better, but every frame of the claimant's home would leave the device, each sweep would cost money per frame, and a network round trip would sit in the live loop. Running locally keeps model cost at zero and the video on the laptop.
- **What this costs.** Small models read fewer worn titles. The design compensates by refusing rather than guessing: a title needs two independent readings, so a weaker model lowers the identification rate but not its honesty.
- **Swapping is configuration only.** On a machine with 16 GB or more, or a GPU, larger models can be set with `BOOK_READER_MODEL`, `CROP_VERIFIER_MODEL` and `CONVERSATION_MODEL` in `backend/.env`; no code changes.

Time budgets per live frame

Stage	Budget	What happens to the rest
Detection (three detectors)	no budget; 1–25 s measured	—
Crop OCR	<code>max(10 s, 1.5 s × spines)</code> , capped at 60 s	spines beyond the budget get a note; retake closer
Title reader (Gemma)	<code>BOOK_READER_FRAME_BUDGET_S</code> = 60 s, best-evidenced crops first	marked <code>deferred</code> , read after capture stops
Blind crop check (Qwen3-VL)	<code>CROP_VERIFICATION_FRAME_BUDGET_S</code> = 90 s, wall frames first	marked <code>deferred</code> , checked after capture stops

The after-capture pass (`state.verify_deferred`) finishes the deferred reads and checks, runs the identity gate on each, and re-finishes the packet; the page shows its progress. Measured on a worn 53-spine shelf: the live frame fell from 799 s to 274 s.

Identity gate

Two evidence-bound paths, both needing a whole spine, a second detector on the same box and the blind check's agreement: `ocr_exact` (the proposal is literally in the OCR text at ≥ 0.9) and `two_models` (the title reader and the blind check, which also transcribes the spine, read the same title and the OCR fragments anchor at least a third of it in order). An author's name is never a title (reader prompt, gate rule, Open Library author search at finish). A title word the blind check read differently is refused as a probable misread, and the Open Library catalogue confirms or flags every accepted title at finish. Evidence is pooled across the frames a spine appears in, with book tracks aligned by the scene transform between views.

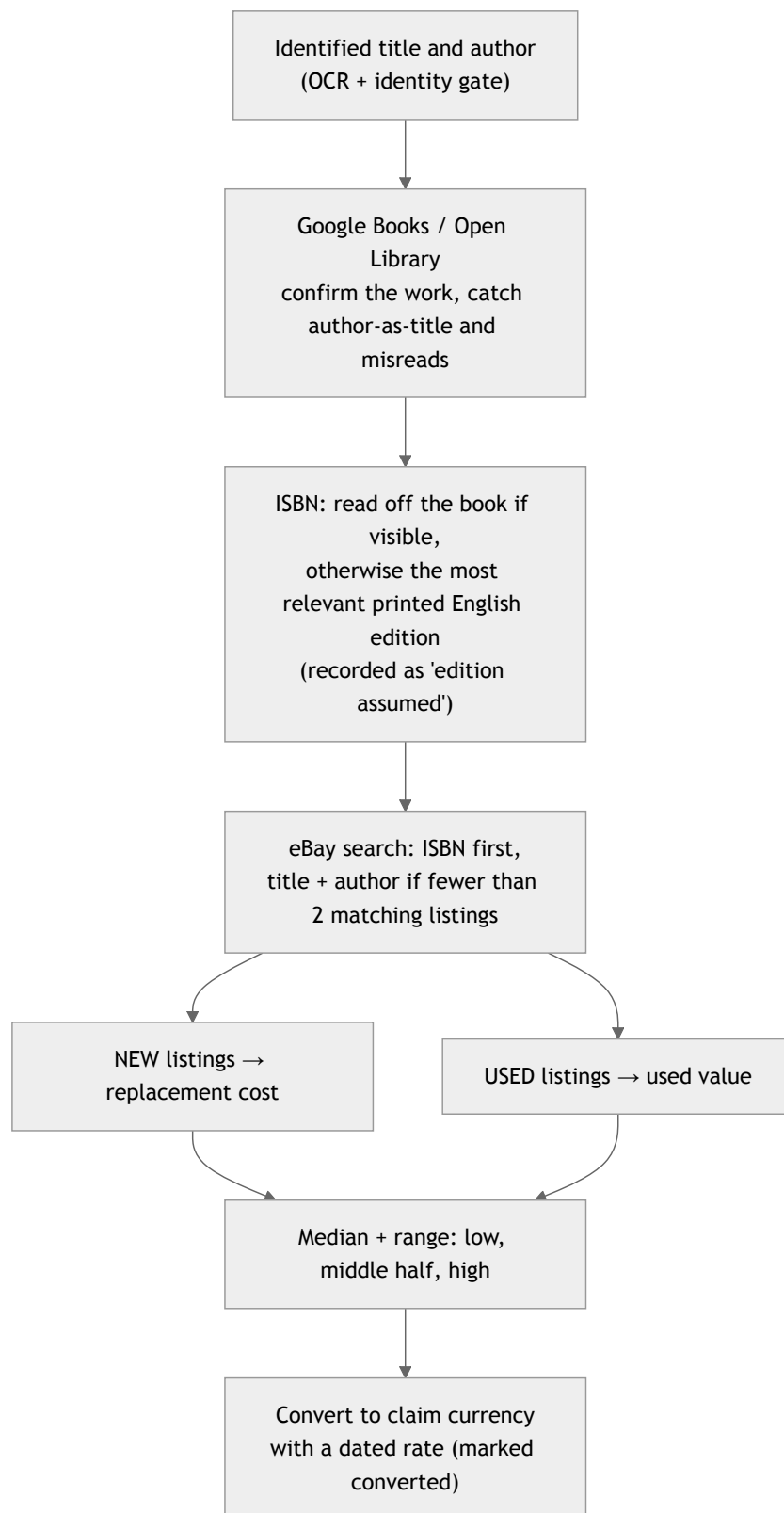
Frontend

Two pages in one React app (`frontend/src/App.tsx`): the sweep page (controls, camera with a status strip, live counts, chat with the agent) and the inventory page (`#inventory` : claim packet links and summary, then every line), enabled once the packet is finished. The layout is responsive from 360 px phones to wide desktops; the dev server is HTTPS so phones on the LAN get the camera and microphone.

Metric scale

- **Spines:** the claimant states a known shelf width during the sweep ("the shelf is 90 centimetres wide"); the detected shelf/book-row span in that frame gives cm per pixel, and the front-on book boxes become spine height $\times$ thickness. An explicit reference line can also be posted to `/calibration` . Method and calibration are stored on each book.
- **Room:** recorded dimensions (tape, plan or LiDAR export) against a sweep frame; rectangle or floor-plan polygon (shoelace area, perimeter $\times$ height for gross walls, shelving rectangles for shelved wall area). `room.scale_method` , `source` , `frame_ref` , `confidence` and assumptions are stored. No separate measuring pass.

Where prices come from



`logic/providers.py` holds the retrieval paths: `resolve_catalogue_isbn` (Google Books then Open Library; a printed edition whose title and author match, English-language first, stored as `catalogue_isbn` with `isbn_source` "edition assumed"), eBay Browse (fixed-price listings, up to 50 per search; ISBN first with title + author as fallback; new → replacement, used → used value; median of matching listings, comparables kept and a `price_range` of count, low, quartiles and high), Google Books (country list prices; ebooks flagged) and FX (dated ECB/ER-API rates for `converted: true`; the range is converted with the same rate). eBay's Browse API has no sold listings, so prices are

labelled as asking prices. `logic/pricing.py` applies the rules without network or models and records every candidate and rejection in `price_details`. Operator-checked offers entered in the review panel carry the same evidence fields. Appraisal rules: signed/rare/antiquarian/ first editions, original art (unless the claimant says "print"), and anything at or above the configurable threshold.

Validation, review queue and totals

`logic/claim.py` rebuilds workflow findings on every change: unreadable spine, confidence below threshold, no accepted price (with the provider rejection reasons), missing source URL/date, currency mismatch, converted price without FX evidence, implausible dimensions, duplicate IDs, missing evidence frame, unknown room scale, unconfirmed coverage. Perception notes (count disagreement, OCR unavailable) and claimant statements are kept. `logic/claim.py` (totals) sums only known values; `excluded_from_totals` counts the blanks.

Latency and cost

Each stage records elapsed seconds (`pipeline_stages` per frame, `stage_runs` per workflow pass); `GET /api/sweeps/{id}/performance` summarises calls, mean and max per stage, and `time_to_packet_s` is measured from stop-capture to finish. Model API cost is zero (local inference); provider calls (eBay, Google Books, FX) are counted in `cost.provider_calls` and are free-tier.